

206. Reverse Linked List Easy

Acceptance: 80.1% | [LeetCode](#)

Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:

```
**Input:** head = [1,2,3,4,5]
**Output:** [5,4,3,2,1]
```

Example 2:

```
**Input:** head = [1,2]
**Output:** [2,1]
```

Example 3:

```
**Input:** head = []
**Output:** []
```

Constraints:

- The number of nodes in the list is the range `[0, 5000]`.
- `-5000 <= Node.val <= 5000`

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

Claude Opus Solution

Reverse Linked List — Complete Solution

1. Intuition

The key insight is that **reversing a linked list means flipping the direction of every `next` pointer**.

In the original list: `1 -> 2 -> 3 -> None` In the reversed list: `None <- 1 <- 2 <- 3` (so `3 -> 2 -> 1 -> None`)

The challenge with singly linked lists is that each node only knows about its *next* node, not its *previous* one. So as we walk forward and flip pointers, we must **remember the previous node** to point back to it. We also must **save the next node before overwriting it**, otherwise we lose access to the rest of the list.

This is the classic **three-pointer / pointer-rewiring pattern** (`previous`, `current`, `next`).

2. Approach (Pseudocode)

```
function reverseList(head):
    previous = None           # nothing comes before the first node yet
    current = head

    while current is not None:
        next_temp = current.next # 1. save the next node (don't lose the rest)
        current.next = previous # 2. reverse the pointer (point backwards)
        previous = current      # 3. move 'previous' forward
        current = next_temp     # 4. move 'current' forward

    return previous           # 'previous' is the new head
```

Why `previous` is the answer: When the loop ends, `current` has walked off the end (`None`), and `previous` is sitting on the last original node — which is now the first node of the reversed list.

3. Python Solution

Iterative (recommended — O(1) space)

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # 'previous' tracks the node that should come AFTER 'current'
        # in the reversed list. Initially None, since the original head
```

```

# becomes the new tail (pointing to None).
previous = None
current = head

# Walk through the list one node at a time.
while current:
    # STEP 1: Save the next node BEFORE we overwrite current.next,
    # otherwise we'd lose the rest of the list permanently.
    next_node = current.next

    # STEP 2: Flip the pointer – current now points backward
    # to whatever came before it.
    current.next = previous

    # STEP 3: Advance 'previous' to the current node, since
    # for the next iteration, current becomes "the previous".
    previous = current

    # STEP 4: Advance 'current' to the saved next node.
    current = next_node

# 'previous' ends up on the last original node = new head.
return previous

```

Note: The reference solution is functionally identical — it just reuses `head` as its moving pointer instead of a separate `current` variable. The version above uses an explicit `current` for clarity.

Recursive (good for the follow-up)

```

class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # BASE CASE: empty list or single node – already "reversed".
        if head is None or head.next is None:
            return head

        # RECURSE: reverse everything AFTER head.
        # 'new_head' is the head of the fully reversed sublist,
        # and it never changes as recursion unwinds.
        new_head = self.reverseList(head.next)

        # At this point: head -> head.next, and the part after
        # head.next is already reversed. We need head.next to point
        # BACK to head.
        # Example: ... 1 -> 2 -> [reversed rest]
        # head = 1, head.next = 2.
        # head.next.next = head -> makes 2 -> 1
        head.next.next = head

        # Sever head's old forward pointer so it becomes the new tail.
        head.next = None

        # Propagate the new head all the way up the call stack.
        return new_head

```

4. Complexity Analysis

Approach	Time	Space	Explanation
Iterative	$O(n)$	$O(1)$	Visit each of the <code>n</code> nodes exactly once; only 3 pointers used regardless of list size.
Recursive	$O(n)$	$O(n)$	Each node triggers one recursive call. The call stack grows to depth <code>n</code> (up to 5000 here, which is safe in Python).

- **Iterative is preferred** for production: constant extra memory and no risk of stack overflow.
- **Recursive is elegant** and great to demonstrate for the follow-up, but consumes stack space proportional to list length.

5. Alternative Approaches

1. **Stack-based:** Push all nodes onto a stack, then pop them to rebuild the list in reverse order. This works but uses **$O(n)$ space** and an extra pass — strictly worse than the iterative pointer method.

2. **Collect values & rebuild:** Store all `val`s in an array, reverse it, then create a new linked list. Uses $O(n)$ space and creates new nodes (doesn't reverse *in place*). Generally discouraged because it doesn't reuse existing nodes.
3. **Recursion (shown above):** A clean way to think about the problem top-down, satisfying the follow-up's "implement both" request.

Key Takeaways

- The **three-pointer technique** (`previous`, `current`, `next`) is *the* canonical pattern for in-place linked list reversal — it shows up in many harder problems (reverse in k-groups, reverse between positions, palindrome check).
- **Always save the next pointer before rewiring**, or you'll lose the rest of the list.
- The original head becomes the new tail; the original tail becomes the new head.

Codex Solution

1. Intuition

A singly linked list only has pointers going **forward**:

```
1 -> 2 -> 3 -> 4 -> 5 -> None
```

To reverse it, we need to make each node point to its **previous** node instead of its next node:

```
None <- 1 <- 2 <- 3 <- 4 <- 5
```

The key pattern is:

Walk through the list once, and for each node, reverse its `next` pointer.

To do this safely, we need to keep track of three things:

1. `prev` — the previous node, initially `None`
2. `curr` — the current node we are processing
3. `next_node` — the original next node, saved before changing `curr.next`

2. Approach

Iterative pseudocode

```
function reverseList(head):
    prev = None
    curr = head

    while curr is not None:
        next_node = curr.next    // Save next node
        curr.next = prev        // Reverse pointer
        prev = curr              // Move prev forward
        curr = next_node        // Move curr forward

    return prev
```

Why return `prev`?

At the end:

- `curr` becomes `None`
- `prev` points to the new head of the reversed list

3. Python Solution

```
from typing import Optional

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val: int = 0, next: Optional["ListNode"] = None):
```

```

        self.val = val
        self.next = next

class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        """
        Reverse a singly linked list iteratively.

        Example:
            Input:  1 -> 2 -> 3 -> None
            Output: 3 -> 2 -> 1 -> None
        """

        # prev will eventually become the new head.
        # Initially, there is no previous node before the head.
        prev = None

        # curr is the node we are currently processing.
        curr = head

        # Traverse the entire linked list.
        while curr is not None:
            # Save the next node before changing curr.next.
            # If we do not save it, we lose access to the rest of the list.
            next_node = curr.next

            # Reverse the pointer.
            # Instead of pointing forward, curr now points backward.
            curr.next = prev

            # Move prev one step forward.
            prev = curr

            # Move curr one step forward using the saved next node.
            curr = next_node

        # When curr becomes None, prev is at the new head.
        return prev

```

4. Complexity Analysis

Let n be the number of nodes in the linked list.

Time Complexity: $O(n)$

We visit each node exactly once.

Each operation inside the loop is $O(1)$, so the total time is:

$O(n)$

Space Complexity: $O(1)$

The iterative solution only uses a few extra pointers:

- prev
- curr
- next_node

No extra data structures are used, so the space complexity is:

$O(1)$

5. Alternative Approaches

Recursive Solution

We can also reverse the linked list recursively.

The idea is:

1. Reverse the rest of the list after `head`
2. Make `head.next.next` point back to `head`
3. Set `head.next = None`
4. Return the new head

Recursive Python Solution

```
from typing import Optional

class SolutionRecursive:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        """
        Reverse a singly linked list recursively.
        """

        # Base case:
        # If the list is empty or has only one node,
        # it is already reversed.
        if head is None or head.next is None:
            return head

        # Recursively reverse the list starting from head.next.
        # new_head will be the head of the reversed list.
        new_head = self.reverseList(head.next)

        # At this point:
        #
        # Original:
        # head -> next_node -> ...
        #
        # After recursion:
        # ... -> next_node -> None
        #
        # We want next_node to point back to head.
        head.next.next = head

        # head is now the last node, so its next must be None.
        head.next = None

        # Return the head of the reversed list.
        return new_head
```

Recursive Complexity

Time Complexity: $O(n)$

Each node is processed once.

Space Complexity: $O(n)$

The recursive call stack can contain up to n calls, so it uses $O(n)$ extra space.

Key Pattern

This problem is a classic example of **pointer manipulation** in linked lists.

The main trick is:

```
next_node = curr.next
curr.next = prev
```

Always save the next node before reversing the pointer, otherwise the rest of the list becomes unreachable.